Praktikum 3
Membuat Program Aplikasi Perceptron

NAMA	: Ihsanta Zaki Sanjaya
KELAS	: 2 D4 Teknik Komputer B
NRP	: 3223600058
DOSEN	: Riyanto Sigit, ST, M.Kom, Ph.D
MATA KULIAH	: Praktikum Sistem Cerdas
TANGGAL	: Rabu, 5 Maret 2025

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

Praktikum 3
Membuat Program Aplikasi Perceptron

A. Tujuan

1. Mahasiswa dapat memahami dan menjelaskan konsep Perceptron
2. Mahasiswa dapat menjelaskan model Perceptron
3. Mahasiswa dapat membuat aplikasi

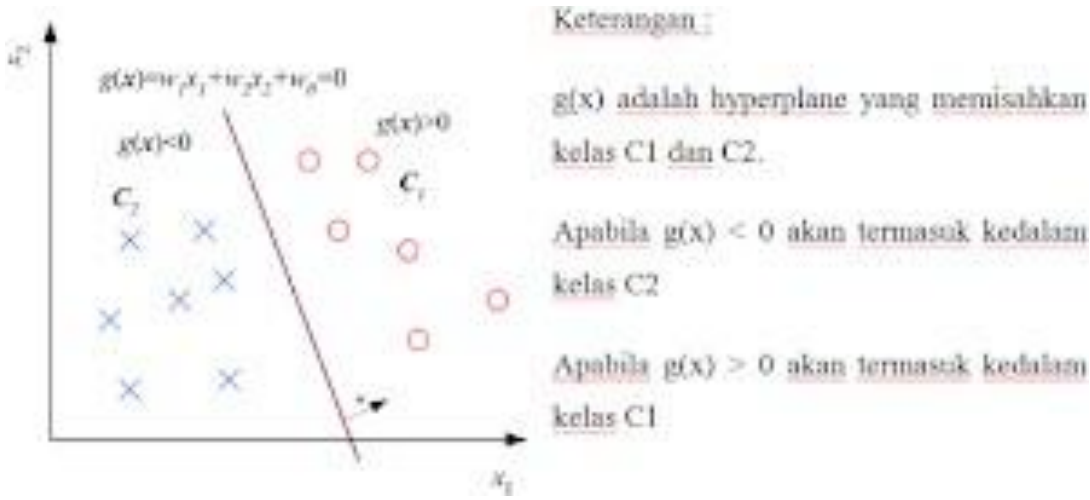
Perceptron Software yang diperlukan:

- Microsoft Visual C++
- PyCharm

B. Pendahuluan

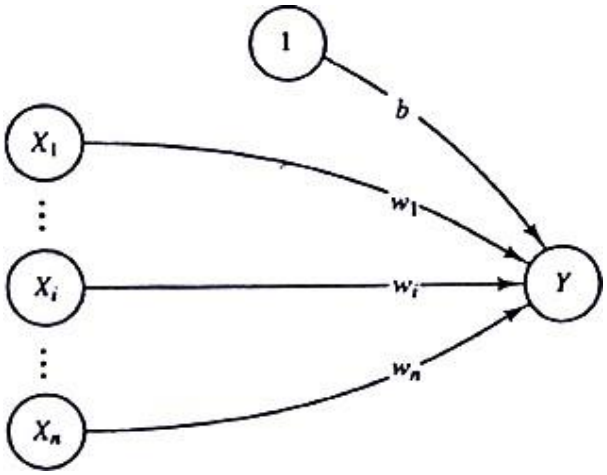
1. Perceptron

Perceptron adalah salah satu metode Jaringan Syaraf Tiruan (JST) sederhana yang menggunakan algoritma training untuk melakukan klasifikasi secara linier. Perceptron digunakan untuk melakukan klasifikasi sederhana dan membagi data untuk menentukan data mana yang masuk dalam klasifikasi dan data mana yang missclasifikasi (diluar klasifikasi). Perceptron dapat kita gunakan untuk memisahkan data yang dapat kita bagi menjadi 2 kelas, misalnya kelas C1 dan kelas C2. Perceptron dalam Jaringan Syaraf Tiruan memiliki bobot yang bisa diatur dalam suatu ambang batas (threshold). Melalui proses pembelajaran (learning), Algotirma Perceptron akan mengatur parameter-parameter bebasnya.



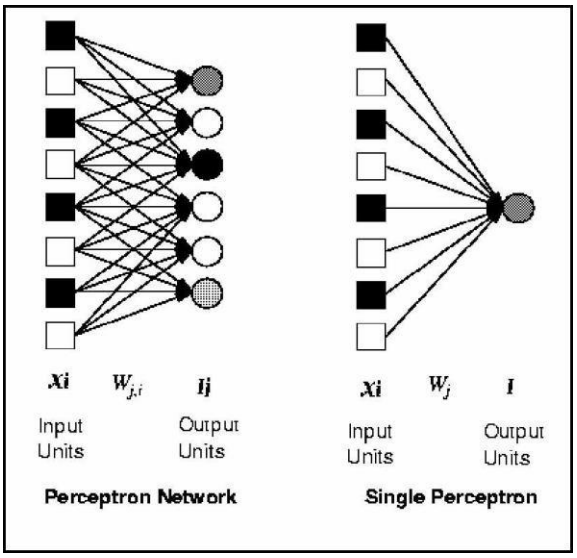
Gambar 1. Klasifikasi Perceptron

Tanda x (biru) dan tanda o (merah) pada gambar diatas dicontohkan adalah dua buah data yang akan diklasifikasi. Dimana kedua data tersebut akan dipisahkan dengan suatu garis hyperplane yang membagi data menjadi dua klasifikasi. Dimana (g(x)) merupakan vektor hyperplane yang memisahkan kelas C1 (data x) dan C2 (data o). Apabila nilai g(x) < 0 (g(x) lebih kecil dari nol) maka data tersebut termasuk dalam kelas C2, sedangkan apabila g(x) > 0 (g(x) lebih besar dari nol) maka data tersebut termasuk kedalam kelas C1.



Gambar 2. Model Perseptron

Jaringan lapis tunggal Perseptron (single layer perceptron) terdiri dari beberapa unit pemroses (neuron), yang terhubung dan mempunyai beberapa masukan serta sebuah atau beberapa keluaran. Single Perceptron sering disebut juga dengan Perceptron.



Gambar 3. Jenis Perceptron

Perseptron menghitung jumlah nilai perkalian penimbang dan masukan dari parameter permasalahan yang kemudian dibandingkan dengan nilai treshold. Bila nilai keluaran lebih besar dari treshold maka keluarannya adalah satu, sebaliknya adalah nol. Jenis perceptron yaitu single output dan multi ouput.

2. Algoritma Perceptron

1. Inisialisasi semua input, matriks vector, target, dan learning rate.
2. Kita lakukan pengecekan untuk semua vector yang misclassification dengan menggunakan rumus berikut :

$$n = \sum x_i w_i + b$$

Jaringan terdiri dari beberapa unit masukan (ditambah sebuah bias), dan memiliki sebuah unit keluaran. Hanya saja fungsi aktivasi merupakan fungsi biner (atau bipolar), tetapi memiliki kemungkinan nilai -1, 0 atau 1.

Dengan ketentuan untuk suatu harga threshold θ yang ditentukan sebagai berikut :

$$a = f(n) = \begin{cases} 1 & n > \theta \\ 0 & -\theta \le n \le \theta \\ -1 & n < -\theta \end{cases}$$

3. Jika hasilnya bernilai negatif, maka terjadi miss classification dan kalikan hasilnya dengan negatif learning rate. Kemudiah hasilnya disimpan dalam bentuk matriks untuk seluruh inputan yang miss clasification.
4. Kemudian lakukan perubahan bobot yang baru dengan menggunakan rumus :

$$W_i(baru) = w_i(lama) + \Delta w(i = 1, \dots, n) dengan\ \Delta w = \alpha t x_i$$

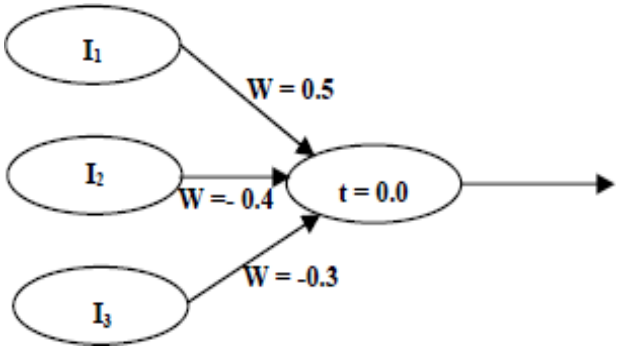
$$b(baru) = b(lama) + \Delta b = \alpha t$$

5. Kemudian ulangi langkah 2 hingga tidak terdapat lagi vector misclassification.

Pelatihan pada perseptron dilakukan dengan merubah nilai penimbangnya sehingga sesuai dengan kebutuhan yang dilakukan dengan membandingkan keluaran dari jaringan dengan targetnya dan proses tersebut dituliskan seperti berikut ini:

$$Wbaru_{ji} = Wlama_{ji} + a(t_j - O)x_i$$

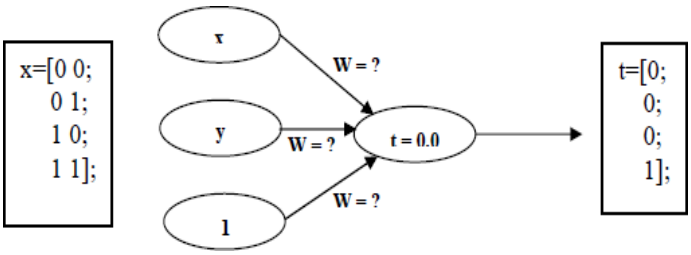
t_j adalah target, O adalah output dari jaringan dan α adalah bilangan konstan bernilai kecil antara 0,1 sampai 0,9 yang disebut sebagai laju pelatihan (learning rate). Mula-mula dilakukan random pada bobot (W) kemudian dilakukan perhitungan perkalian antara input (I) dan bobot (W) seperti:



I_1	I_2	I_3	Penjumlahan	Output
0	0	1	(0*0.5) + (0*-0.4) + (1*-0.3) = -0.3	0
0	1	1	(0*0.5) + (1*-0.4) + (1*-0.3) = -0.7	0
1	0	1	(1*0.5) + (0*-0.4) + (1*-0.3) = 0.2	1
1	1	1	(1*0.5) + (1*-0.4) + (1*-0.3) = -0.2	0

Gambar 4. Perhitungan Pelatihan Perseptron

Output diperoleh dengan melakukan fungsi aktifasi dalam kasus AND dilakukan dengan unit step yaitu bila kurang dari nol output = 0, bila lebih output =1. Error diperoleh apabila terjadi perbedaan antara target dengan output jaringan. Jika error tidak sama dengan nol maka bobot (W) diperbaiki. $Wbaru_{ji} = Wlama_{ji} + \alpha(t_j\text{-}O)x_i$



Gambar 5. Error diperoleh apabila terjadi perbedaan target dan output jaringan

C. Percobaan

1. Logika AND

AND

X_1	X_2	Y
0	0	0
0	1	0
1	0	0
1	1	1

```
double d_rand(){
    return(((dodoubleuble)(rand()%32767)/32767.0-
    0.5)*2.0);
}
void main()
{
    int
    i,j,out,ERR,y
    ; int x[3][4]=
    {
        {0,0,1,1},
        {0,1,0,1},
        {1,1,1,1}
    };
    int T[4]={0,0,0,1};
    float w[3],O,LR=0.1,init=0.15;
    //inisialisasi
    bobot
    for(i=0;i<3;i+
    +){
        w[i]=init*d_rand();
    }
    //traning
    for(i=0;i<10;i++
    ){
        for(j=0;j<4;j++){
            O=x[0][j]*w[0]+x[1][j]*w[1]+x[2][j]*w[2];
            if(O>0.0)
                out
            =1;
            else
                out=0;
            ERR=T[j]-
            out;
            if(ERR!=0){
                w[0]=w[0]+LR*x[0][j]*ERR;
                w[1]=w[1]+LR*x[1][j]*ERR;
                w[2]=w[2]+LR*x[2][j]*ERR;
            }
        }
        printf("Err:%d\n",ERR);
    }
    //running
    for(i=0;i<4;i+
    +){
        printf("Input X1:");
        scanf("%d",&x[0][0]);
        printf("Input X2:");
        scanf("%d",&x[1][0]);
        O=x[0][0]*w[0]+x[1][0]*w[1]+x[2][0]*w[2];
        if(O>0.0)
            y=1;
        else
            y=0;
```

```
    printf("Output:%d\n",y);
  }
}
```

Hasil Program

```
"D:\PENS\Tekkom\Sistem Cerdas\2020 Sistem Cerdas\Praktikum Sistem Cerc
Err:1
Err:1
Err:1
Err:1
Err:1
Err:1
Err:0
Err:0
Err:0
Err:0
Input X1:0
Input X2:0
Output:0
Input X1:0
Input X2:1
Output:0
Input X1:1
Input X2:0
Output:0
Input X1:1
Input X2:1
Output:1

Process returned 0 (0x0)   execution time : 10.503 s
Press any key to continue.
```

2. Logika OR

OR

X_1	X_2	Y
0	0	0
0	1	1
1	0	1
1	1	1

```
double d_rand(){
    return(((double)(rand()%32767)/32767.0-
    0.5)*2.0);
}
void main()
{
    int
    i,j,out,ERR,y
    ; int x[3][4]=
    {
        {0,0,1,1},
        {0,1,0,1},
        {1,1,1,1}
    };
    int T[4]={0,1,1,1};
    float w[3],O,LR=0.1,init=0.15;
    //inisialisasi
    bobot
    for(i=0;i<3;i+
    +){
        w[i]=init*d_rand();
    }
    //traning
    for(i=0;i<10;i++){
        for(j=0;j<4;j++){
            O=x[0][j]*w[0]+x[1][j]*w[1]+x[2][j]*w[2];
            if(O>0.0)
                out=1;
            else
                out=0;
            ERR=T[j]-
            out;
            if(ERR!=0){
                w[0]=w[0]+LR*x[0][j]*ERR;
                w[1]=w[1]+LR*x[1][j]*ERR;
                w[2]=w[2]+LR*x[2][j]*ERR;
            }
        }
        printf("Err:%d\n",ERR);
    }
    //running
    for(i=0;i<4;i++){
        printf("Input X1:");
        scanf("%d",&x[0][
        0]); printf("Input
        X2:");
        scanf("%d",&x[1][
        0]);
        O=x[0][0]*w[0]+x[1][0]*w[1]+x[2][0]*w[2];
        if(O>0.0)
            y=1;
        else
            y=0;
        printf("Output:%d\n",y);
```

```
    }
  }
```

Hasil Program

```
"D:\PENS\Tekkom\Sistem Cerdas\2020 Sistem Cerdas\Praktikum Sistem Cer
Err:0
Err:0
Err:0
Err:0
Err:0
Err:0
Err:0
Err:0
Err:0
Err:0
Input X1:0
Input X2:0
Output:0
Input X1:0
Input X2:1
Output:1
Input X1:1
Input X2:0
Output:1
Input X1:1
Input X2:1
Output:1

Process returned 0 (0x0)   execution time : 10.299 s
Press any key to continue.
```

D. Hasil Percobaan

1. Perceptron AND

Program:

perceptron_and.cpp > main()

```cpp
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
double d_rand(){
    return(((double)(rand()%32767)/32767.0-0.5)*2.0);
}
int main() {
    int i,j,out,ERR,y;
    int x[3][4]= {
        {0,0,1,1},
        {0,1,0,1},
        {1,1,1,1}
    };
    int T[4]={0,0,0,1};
    float w[3],O,LR=0.1,init=0.15;
    //inisialisasi bobot
    for(i=0;i<3;i++){
        w[i]=init*d_rand();
    }
    //traning
    for(i=0;i<10;i++){
        for(j=0;j<4;j++){
            O=x[0][j]*w[0]+x[1][j]*w[1]+x[2][j]*w[2];
            if(O>0.0)
                out=1;
            else
                out=0;
            ERR=T[j]-out;
            if(ERR!=0){
                w[0]=w[0]+LR*x[0][j]*ERR;
                w[1]=w[1]+LR*x[1][j]*ERR;
                w[2]=w[2]+LR*x[2][j]*ERR;
            }
        }
        printf("Err:%d\n",ERR);
    }
    //running
    for(i=0;i<4;i++){
        printf("Input X1:");
        scanf("%d",&x[0][0]);
        printf("Input X2:");
        scanf("%d",&x[1][0]);
        O=x[0][0]*w[0]+x[1][0]*w[1]+x[2][0]*w[2];
        if(O>0.0)
            y=1;
        else
            y=0;
        printf("Output:%d\n",y);
    }
    return 0;
}
```

Output:

```
Err:1
Err:1
Err:1
Err:1
Err:1
Err:1
Err:0
Err:0
Err:0
Err:0
Input X1:0
Input X2:0
Output:0
Input X1:0
Input X2:1
Output:0
Input X1:1
Input X2:0
Output:0
Input X1:1
Input X2:1
Output:1
```

Truth table AND:

x1	x2	Output
0	0	0
0	1	0
1	0	0
1	1	1

2. Perceptron OR

Program:

perceptron_or.cpp > main()

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
double d_rand(){
    return(((double)(rand()%32767)/32767.0-0.5)*2.0);
}
int main() {
    int i,j,out,ERR,y;
    int x[3][4]= {
        {0,0,1,1},
        {0,1,0,1},
        {1,1,1,1}
    };
    int T[4]={0,1,1,1};
    float w[3],O,LR=0.1,init=0.15;
    //inisialisasi bobot
    for(i=0;i<3;i++){
        w[i]=init*d_rand();
    }
    //traning
    for(i=0;i<10;i++){
        for(j=0;j<4;j++){
            O=x[0][j]*w[0]+x[1][j]*w[1]+x[2][j]*w[2];
            if(O>0.0)
                out=1;
            else
                out=0;
            ERR=T[j]-out;
            if(ERR!=0){
                w[0]=w[0]+LR*x[0][j]*ERR;
                w[1]=w[1]+LR*x[1][j]*ERR;
                w[2]=w[2]+LR*x[2][j]*ERR;
            }
        }
        printf("Err:%d\n",ERR);
    }
    //running
    for(i=0;i<4;i++){
        printf("Input X1:");
        scanf("%d",&x[0][0]);
        printf("Input X2:");
        scanf("%d",&x[1][0]);
        O=x[0][0]*w[0]+x[1][0]*w[1]+x[2][0]*w[2];
        if(O>0.0)
            y=1;
        else
            y=0;
        printf("Output:%d\n",y);
    }
    return 0;
}
```

Output:

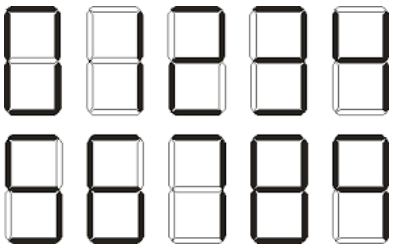
```
Err:0
Err:0
Err:0
Err:0
Err:0
Err:0
Err:0
Err:0
Err:0
Err:0
Input X1:0
Input X2:0
Output:0
Input X1:0
Input X2:1
Output:1
Input X1:1
Input X2:0
Output:1
Input X1:1
Input X2:1
Output:1
```

Truth table OR:

x1	x2	Output
0	0	0
0	1	1
1	0	1
1	1	1

E. Tugas

1. Buat program untuk 7 segment input 7 output 10



Program:

perceptron_7segment.cpp > ...

```cpp
#include <stdio.h>
#include <stdlib.h>
double d_rand() {
    return (((double)(rand() % 32767) / 32767.0 - 0.5) * 2.0);
}
int main() {
    int i, j, k, out, ERR, y;
    int x[10][7] = {
        {1, 1, 1, 1, 1, 1, 0}, // 0
        {0, 1, 1, 0, 0, 0, 0}, // 1
        {1, 1, 0, 1, 1, 0, 1}, // 2
        {1, 1, 1, 1, 0, 0, 1}, // 3
        {0, 1, 1, 0, 0, 1, 1}, // 4
        {1, 0, 1, 1, 0, 1, 1}, // 5
        {1, 0, 1, 1, 1, 1, 1}, // 6
        {1, 1, 1, 0, 0, 0, 0}, // 7
        {1, 1, 1, 1, 1, 1, 1}, // 8
        {1, 1, 1, 1, 0, 1, 1}};  // 9
    int T[10][10] = {
        {1, 0, 0, 0, 0, 0, 0, 0, 0, 0}, // 0
        {0, 1, 0, 0, 0, 0, 0, 0, 0, 0}, // 1
        {0, 0, 1, 0, 0, 0, 0, 0, 0, 0}, // 2
        {0, 0, 0, 1, 0, 0, 0, 0, 0, 0}, // 3
        {0, 0, 0, 0, 1, 0, 0, 0, 0, 0}, // 4
        {0, 0, 0, 0, 0, 1, 0, 0, 0, 0}, // 5
        {0, 0, 0, 0, 0, 0, 1, 0, 0, 0}, // 6
        {0, 0, 0, 0, 0, 0, 0, 1, 0, 0}, // 7
        {0, 0, 0, 0, 0, 0, 0, 0, 1, 0}, // 8
        {0, 0, 0, 0, 0, 0, 0, 0, 0, 1}};  // 9
    float w[7][10], O[10], LR = 0.1, init = 0.15;
    for (i = 0; i < 7; i++) {
        for (j = 0; j < 10; j++) {
            w[i][j] = init * d_rand();
        }
    }
    int count = 0;
    for (k = 0; k < 50; k++) {
        for (i = 0; i < 10; i++) {
            for (j = 0; j < 10; j++) {
                O[j] = 0;
                for (int l = 0; l < 7; l++) {
                    O[j] += x[i][l] * w[l][j];
                }
                out = (O[j] > 0.0) ? 1 : 0;
                ERR = T[i][j] - out;
                if (ERR != 0) {
                    for (int l = 0; l < 7; l++) {
                        w[l][j] += LR * x[i][l] * ERR;
                    }}}}
        printf("Epoch %d ERR = %d\n", count, ERR); // Hanya mengubah printf
        count++;}
    int input[7];
    printf("Masukkan input 7-segment (7 angka 0 atau 1):\n");
    for (i = 0; i < 7; i++) {
        scanf("%d", &input[i]);
    }
    for (j = 0; j < 10; j++) {
        O[j] = 0;
        for (i = 0; i < 7; i++) {
            O[j] += input[i] * w[i][j];
        }
        out = (O[j] > 0.0) ? 1 : 0;
        if (out == 1) {
            printf("Output: %d\n", j);
            break;
        }
    }
    return 0;
}
```

Output:

```
Epoch 1 ERR = 1
Epoch 2 ERR = 1
Epoch 3 ERR = 1
Epoch 4 ERR = 1
Epoch 5 ERR = 1
Epoch 6 ERR = 1
Epoch 7 ERR = 1
Epoch 8 ERR = 1
Epoch 9 ERR = 1
Epoch 10 ERR = 1
Epoch 11 ERR = 1
Epoch 12 ERR = 1
Epoch 13 ERR = 1
Epoch 14 ERR = 1
Epoch 15 ERR = 1
Epoch 16 ERR = 1
Epoch 17 ERR = 1
Epoch 18 ERR = 1
Epoch 19 ERR = 1
Epoch 20 ERR = 1
Epoch 21 ERR = 1
Epoch 22 ERR = 1
Epoch 23 ERR = 1
Epoch 24 ERR = 1
Epoch 25 ERR = 1
Epoch 26 ERR = 1
Epoch 27 ERR = 1
Epoch 28 ERR = 0
Epoch 29 ERR = 0
Epoch 30 ERR = 0
Epoch 31 ERR = 0
Epoch 32 ERR = 0
Epoch 33 ERR = 0
Epoch 34 ERR = 0
Epoch 35 ERR = 0
Epoch 36 ERR = 0
Epoch 37 ERR = 0
Epoch 38 ERR = 0
Epoch 39 ERR = 0
Epoch 40 ERR = 0
Epoch 41 ERR = 0
Epoch 42 ERR = 0
Epoch 43 ERR = 0
Epoch 44 ERR = 0
Epoch 45 ERR = 0
Epoch 46 ERR = 0
Epoch 47 ERR = 0
Epoch 48 ERR = 0
Epoch 49 ERR = 0
Masukkan input 7-segment (7 angka 0 atau 1):
```

```
Masukkan input 7-segment (7 angka 0 atau 1):
1
1
1
1
1
1
0
Output: 0
Masukkan input 7-segment (7 angka 0 atau 1):
0
1
1
0
0
0
0
Output: 1
Masukkan input 7-segment (7 angka 0 atau 1):
1
1
0
1
1
0
1
Output: 2
Masukkan input 7-segment (7 angka 0 atau 1):
1
1
1
1
0
0
1
Output: 3
Masukkan input 7-segment (7 angka 0 atau 1):
0
1
1
0
0
1
1
Output: 4
Masukkan input 7-segment (7 angka 0 atau 1):
1
0
1
1
0
1
1
Output: 5
```

```
Masukkan input 7-segment (7 angka 0 atau 1):
1
1
1
0
0
0
0
Output: 7
Masukkan input 7-segment (7 angka 0 atau 1):
1
1
1
1
1
1
1
Output: 8
Masukkan input 7-segment (7 angka 0 atau 1):
1
1
1
1
0
1
1
Output: 9
```

Truth table Seven Segment

A	B	C	D	E	F	G	Display
1	1	1	1	1	1	0	0
0	1	1	0	0	0	0	1
1	1	0	1	1	0	1	2
1	1	1	1	0	0	1	3
0	1	1	0	0	1	1	4
1	0	1	1	0	1	1	5
1	0	1	1	1	1	1	6
1	1	1	0	0	0	0	7
1	1	1	1	1	1	1	8
1	1	1	1	0	1	1	9

F. Analisa

Pada implementasi operasi logika AND menggunakan perceptron, program ini bekerja dengan tiga input, yaitu dua nilai input utama (X1 dan X2) serta satu bias. Bobot awal diinisialisasi secara acak dalam rentang tertentu dan diperbarui melalui proses pelatihan selama 10 epoch. Selama pelatihan, bobot diperbarui berdasarkan error yang dihitung dari selisih antara keluaran yang dihasilkan dan target yang diinginkan. Jika terjadi error, bobot diperbarui menggunakan aturan pembelajaran perceptron:

```
w[0]=w[0]+LR*x[0][j]*ERR;
w[1]=w[1]+LR*x[1][j]*ERR;
w[2]=w[2]+LR*x[2][j]*ERR;
```

Setelah pelatihan selesai, program memasuki tahap pengujian, di mana pengguna dapat memasukkan nilai X1 dan X2 sebagai input. Program kemudian menghitung output berdasarkan bobot yang telah diperbarui selama pelatihan. Model perceptron ini bekerja dengan baik dalam mengenali pola logika AND, di mana output hanya akan bernilai 1 jika kedua input bernilai 1, dan 0 jika sebaliknya.

Pada implementasi operasi logika OR, metode pelatihan dan pengujian hampir sama dengan program AND. Perbedaan utama terletak pada target output yang digunakan. Program OR memiliki target output yang mencerminkan fungsi logika OR, yaitu 1 jika salah satu atau kedua input bernilai 1, dan 0 hanya jika kedua input bernilai 0. Selama pelatihan, bobot diperbarui berdasarkan error, sehingga model mampu mengenali pola logika OR dengan baik. Dengan perbedaan target output ini, hasil prediksi dari model OR lebih fleksibel dibandingkan model AND, karena dapat menghasilkan output 1 untuk lebih banyak kombinasi input. Seperti halnya program AND, model OR juga menggunakan aturan pembelajaran perceptron untuk memperbarui bobot guna meningkatkan akurasi prediksi, sesuai dengan kode berikut:

```
O=x[0][j]*w[0]+x[1][j]*w[1]+x[2][j]*w[2];
if(O>0.0)
    out=1;
```

Dalam implementasi seven-segment display, program ini bertujuan untuk mengenali angka 0-9 berdasarkan pola segmen yang digunakan pada display tujuh segmen. Program ini menggunakan tujuh input yang mewakili tujuh segmen utama serta satu bias tambahan. Bobot awal diinisialisasi secara acak dan diperbarui selama proses pelatihan yang berlangsung dalam 1000 epoch. Selama pembelajaran, bobot diperbarui berdasarkan error antara prediksi dan target menggunakan rumus berikut:

```
w[l][j] += LR * x[i][l] * ERR;
```

Setelah pelatihan selesai, sistem dapat menerima input dalam bentuk pola biner seven-segment display dan menentukan angka yang paling sesuai berdasarkan bobot yang telah diperbarui. Meskipun model ini dapat mengenali angka dengan baik setelah pelatihan yang cukup, kompleksitasnya lebih tinggi dibandingkan model AND dan OR karena memiliki sepuluh output yang berbeda. Oleh karena itu, diperlukan iterasi yang lebih banyak serta pengaturan learning rate yang optimal agar proses pembelajaran tetap stabil.

G. Kesimpulan

Perceptron dapat menyelesaikan berbagai permasalahan, seperti operasi logika sederhana AND dan OR hingga pengenalan pola yang lebih kompleks seperti seven-segment display. Model perceptron bekerja berdasarkan aturan pembelajaran berbasis error, di mana bobot diperbarui secara iteratif untuk meningkatkan akurasi prediksi. Sehingga perceptron dapat belajar dari target output yang berbeda dan dapat menyelesaikan masalah dalam klasifikasi multi-kelas. Namun tetap terdapat keterbatasan untuk menangani masalah non-linear yang lebih kompleks. Sehingga dibutuhkan aplikasi yang lebih kompleks seperti multi-perceptron.